BrewFlow Final Project

Teil 3 ETL Pipeline

Projektdokumentation zur Übertragung der OLTP-Daten über Staging in das analytische Star-Schema

Ziel und bestätigtes Ergebnis

Die Pipeline lädt die BrewFlow-Quelldaten in drei aufeinanderfolgenden SQL-Skripten in das Warehouse. Die Staging-Ebene wird vollständig erneuert, Dimensionen werden geladen und Bestellpositionen über ihre Dimensionsschlüssel zugeordnet.

Der Erstlauf und ein zweiter Durchlauf auf unveränderten Quelldaten wurden erfolgreich geprüft. Das Warehouse enthält 32 eindeutige Bestellpositionen mit einem Gesamtumsatz von 169,00. Alle 10 Kundenversionen sind aktuell; die Kalenderdimension enthält 4018 Tage.

Architektur und Ausführungsreihenfolge

brewflow_oltp → brewflow_staging → brewflow_warehouse

Skript	Aufgabe
01_copy_to_staging.sql	Staging vollständig aus der OLTP neu laden
02_load_dimensions.sql	Kunden historisieren, Type-1-Attribute aktualisieren und Kalender erzeugen
03_load_facts.sql	Bestellpositionen mit zeitlicher Kundenzuordnung einfügen

Voraussetzung sind die in Teil 1 erstellte OLTP-Datenbank und das in Teil 2 angelegte Warehouse. dim_channel enthält bereits in-store und mobile-app. Nach der einmaligen Staging-Einrichtung werden die drei Ladeskripte in der angegebenen Reihenfolge ausgeführt.

Anpassungen an der Aufgabenvorlage

Die COUNT-Abfragen verwenden COUNT(*). Die erste Kundenversion wird auf das früheste vorhandene Bestelldatum zurückgeführt, damit Bestellungen aus 2024 zugeordnet werden können. Ein eindeutiger Index auf order_item_id ermöglicht den Konfliktschutz beim Faktenladen.

Produkte und Filialen werden per UPDATE und INSERT geladen. Das vermeidet die Löschung bestehender Fakten durch TRUNCATE CASCADE und erhält die Surrogatschlüssel. Die Umsetzung nutzt für Kunden UPDATE plus INSERT und für Fakten INSERT mit ON CONFLICT; ein SQL-MERGE-Befehl wird nicht verwendet.

Schritt 1 Staging erstellen

Aufgabenstellung Eine Puffer-Ebene mit den fünf Quelltabellen bereitstellen.

Schritt Das Schema brewflow_staging und Tabellen nach den Spaltenstrukturen der OLTP erstellen.

Aktion und SQL Die Tabellen werden mit LIKE angelegt. IF NOT EXISTS erlaubt die erneute Ausführung der Einrichtung, prüft aber keine abweichende Struktur bereits vorhandener Tabellen.

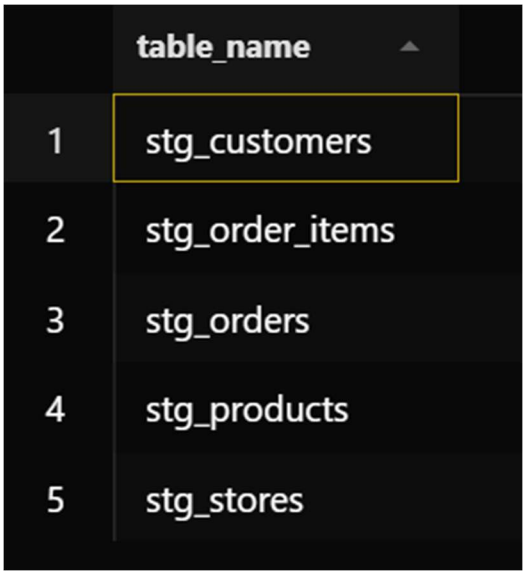
```
-- Teil 3 Schritt 1 Staging erstellen
CREATE SCHEMA IF NOT EXISTS brewflow_staging;
CREATE TABLE IF NOT EXISTS brewflow_staging.stg_customers
    (LIKE brewflow_oltp.customers);
CREATE TABLE IF NOT EXISTS brewflow_staging.stg_stores
    (LIKE brewflow_oltp.stores);
CREATE TABLE IF NOT EXISTS brewflow_staging.stg_products
    (LIKE brewflow_oltp.products);
CREATE TABLE IF NOT EXISTS brewflow_staging.stg_orders
    (LIKE brewflow_oltp.orders);
CREATE TABLE IF NOT EXISTS brewflow_staging.stg_order_items
    (LIKE brewflow_oltp.order_items);

-- Kontrolle Fünf Staging Tabellen
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'brewflow_staging'
  AND table_type = 'BASE TABLE'
ORDER BY table_name;
```

Kontrolle Die Ergebnisliste zeigt stg_customers, stg_order_items, stg_orders, stg_products und stg_stores.

Ergebnis Bestanden. Alle fünf Staging-Tabellen sind vorhanden.

Abbildung 1 Ergebnisnachweis aus dem SQL-Client



Schritt 2 OLTP Daten ins Staging kopieren

Aufgabenstellung Den aktuellen vollständigen Quellbestand in die Staging-Ebene übernehmen.

Schritt Staging leeren und alle fünf Tabellen innerhalb einer Transaktion neu befüllen.

Aktion und SQL Nur dieses Ladeskript liest aus brewflow_oltp. BEGIN und COMMIT fassen die Änderungen zu einer Transaktion zusammen.

```
-- 01_copy_to_staging.sql
-- Staging vollständig aus der OLTP neu laden
BEGIN;
TRUNCATE TABLE
    brewflow_staging.stg_customers,
    brewflow_staging.stg_stores,
    brewflow_staging.stg_products,
    brewflow_staging.stg_orders,
    brewflow_staging.stg_order_items;

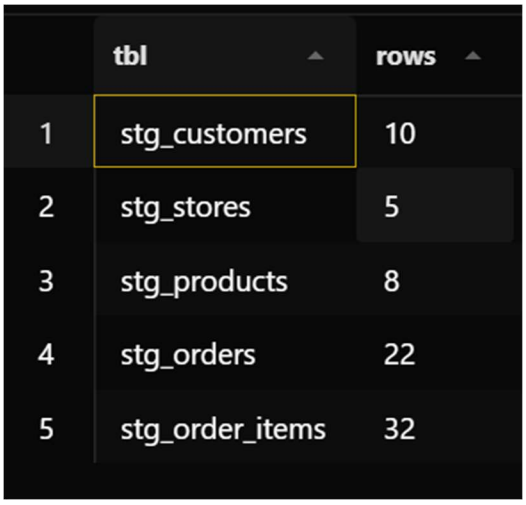
INSERT INTO brewflow_staging.stg_customers
SELECT * FROM brewflow_oltp.customers;
INSERT INTO brewflow_staging.stg_stores
SELECT * FROM brewflow_oltp.stores;
INSERT INTO brewflow_staging.stg_products
SELECT * FROM brewflow_oltp.products;
INSERT INTO brewflow_staging.stg_orders
SELECT * FROM brewflow_oltp.orders;
INSERT INTO brewflow_staging.stg_order_items
SELECT * FROM brewflow_oltp.order_items;
COMMIT;

-- Kontrolle Erwartet 10 / 5 / 8 / 22 / 32
SELECT 'stg_customers' AS tbl, COUNT(*) AS rows
FROM brewflow_staging.stg_customers
UNION ALL SELECT 'stg_stores', COUNT(*)
FROM brewflow_staging.stg_stores
UNION ALL SELECT 'stg_products', COUNT(*)
FROM brewflow_staging.stg_products
UNION ALL SELECT 'stg_orders', COUNT(*)
FROM brewflow_staging.stg_orders
UNION ALL SELECT 'stg_order_items', COUNT(*)
FROM brewflow_staging.stg_order_items;
```

Kontrolle Die fünf Zeilenanzahlen sind 10 Kunden, 5 Filialen, 8 Produkte, 22 Bestellungen und 32 Positionen.

Ergebnis Bestanden. Die Zeilenanzahlen im Staging entsprechen den Vorgaben.

Abbildung 2 Ergebnisnachweis aus dem SQL-Client



	tbl	rows
1	stg_customers	10
2	stg_stores	5
3	stg_products	8
4	stg_orders	22
5	stg_order_items	32

Schritt 3 Dimensionen laden

Aufgabenstellung Die Dimensionen aus dem Staging befüllen und die SCD-Type-2-Struktur für Kunden verwenden.

Schritt Geänderte Loyalty-Versionen beenden, neue Versionen ergänzen, Type-1-Attribute aktualisieren und den Kalender erzeugen.

Aktion und SQL Die beiden folgenden Seiten enthalten zusammen Skript 02. BEGIN steht am Anfang und COMMIT am Ende; beide Seiten werden gemeinsam ausgeführt.

Für den Erstbestand gilt die Annahme, dass das im Staging vorhandene Loyalty-Level bereits am frühesten Bestelldatum galt. Die Quelle enthält keine historische Levelentwicklung. Erst später beobachtete Änderungen werden ab dem Ladetag historisiert.

```
-- 02_load_dimensions.sql
BEGIN;

-- Kunden Geänderte Loyalty Versionen beenden
UPDATE brewflow_warehouse.dim_customer dc
SET effective_to = CURRENT_DATE - 1,
    is_current = FALSE
FROM brewflow_staging.stg_customers src
WHERE dc.customer_id = src.customer_id
  AND dc.is_current = TRUE
  AND dc.loyalty_tier IS DISTINCT FROM src.loyalty_tier;

-- Kunden Neue oder geänderte Versionen ergänzen
INSERT INTO brewflow_warehouse.dim_customer (
    customer_id, first_name, last_name, email, city,
    loyalty_tier, effective_from, effective_to, is_current
)
SELECT
    src.customer_id, src.first_name, src.last_name,
    src.email, src.city, src.loyalty_tier,
    CASE
        WHEN EXISTS (
            SELECT 1 FROM brewflow_warehouse.dim_customer hist
            WHERE hist.customer_id = src.customer_id
        ) THEN CURRENT_DATE
        ELSE COALESCE(
            (SELECT MIN(o.order_date)
             FROM brewflow_staging.stg_orders o
             WHERE o.customer_id = src.customer_id),
            src.joined_date, CURRENT_DATE
        )
    END,
    NULL, TRUE
FROM brewflow_staging.stg_customers src
WHERE NOT EXISTS (
    SELECT 1 FROM brewflow_warehouse.dim_customer dc
    WHERE dc.customer_id = src.customer_id
      AND dc.is_current = TRUE
);

-- Kunden Übrige Attribute nach Type 1 aktualisieren
UPDATE brewflow_warehouse.dim_customer dc
SET first_name = src.first_name,
    last_name = src.last_name,
    email = src.email,
    city = src.city
FROM brewflow_staging.stg_customers src
WHERE dc.customer_id = src.customer_id
  AND dc.is_current = TRUE;
```

Schritt 3 Produkte Filialen und Kalender

```
-- 02_load_dimensions.sql Fortsetzung
-- Produkte Type 1 mit stabilen Surrogatschlüsseln
UPDATE brewflow_warehouse.dim_product dp
SET name = src.name, category = src.category,
    unit_price = src.unit_price
FROM brewflow_staging.stg_products src
WHERE dp.product_id = src.product_id;

INSERT INTO brewflow_warehouse.dim_product (
    product_id, name, category, unit_price
)
SELECT src.product_id, src.name, src.category, src.unit_price
FROM brewflow_staging.stg_products src
WHERE NOT EXISTS (
    SELECT 1 FROM brewflow_warehouse.dim_product dp
    WHERE dp.product_id = src.product_id
);

-- Filialen Type 1 mit stabilen Surrogatschlüsseln
UPDATE brewflow_warehouse.dim_store ds
SET name = src.name, city = src.city, country = src.country
FROM brewflow_staging.stg_stores src
WHERE ds.store_id = src.store_id;

INSERT INTO brewflow_warehouse.dim_store (store_id, name, city, country)
SELECT src.store_id, src.name, src.city, src.country
FROM brewflow_staging.stg_stores src
WHERE NOT EXISTS (
    SELECT 1 FROM brewflow_warehouse.dim_store ds
    WHERE ds.store_id = src.store_id
);

-- dim_channel Bereits in Teil 2 befüllt
-- Kalender 2020 bis 2030 erzeugen
INSERT INTO brewflow_warehouse.dim_date (
    date_sk, calendar_date, year, quarter, month,
    month_name, day_of_week, is_weekend
)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT,
    d::DATE,
    EXTRACT(YEAR FROM d)::INT,
    EXTRACT(QUARTER FROM d)::INT,
    EXTRACT(MONTH FROM d)::INT,
    TO_CHAR(d, 'FMMonth'),
    TO_CHAR(d, 'FMDay'),
    EXTRACT(ISODOW FROM d) IN (6, 7)
FROM GENERATE_SERIES(
    TIMESTAMP '2020-01-01',
    TIMESTAMP '2030-12-31',
    INTERVAL '1 day'
) AS gs(d)
ON CONFLICT (date_sk) DO NOTHING;
COMMIT;
```

Kontrolle Die separate Dimensionsprüfung auf der nächsten Seite vergleicht alle erwarteten Zeilenanzahlen.

Ergebnis Die Dimensionen wurden erfolgreich geladen. Die Umsatzauswertungen erfolgen später über ihre Beziehungen zur Faktentabelle.

Schritt 3 Dimensionen überprüfen

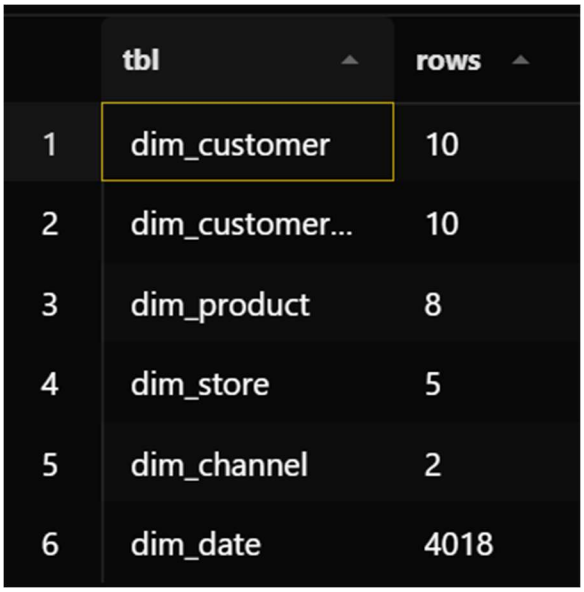
```
-- Skript 02 Kontrolle Dimensionen
-- Erwartet 10 / 10 / 8 / 5 / 2 / 4018
SELECT 'dim_customer' AS tbl, COUNT(*) AS rows
FROM brewflow_warehouse.dim_customer
UNION ALL SELECT 'dim_customer_current', COUNT(*)
FROM brewflow_warehouse.dim_customer WHERE is_current = TRUE
UNION ALL SELECT 'dim_product', COUNT(*)
FROM brewflow_warehouse.dim_product
UNION ALL SELECT 'dim_store', COUNT(*)
FROM brewflow_warehouse.dim_store
UNION ALL SELECT 'dim_channel', COUNT(*)
FROM brewflow_warehouse.dim_channel
UNION ALL SELECT 'dim_date', COUNT(*)
FROM brewflow_warehouse.dim_date;
```

Kontrolle	Bestätigter Wert
dim_customer	10
dim_customer_current	10
dim_product	8
dim_store	5
dim_channel	2
dim_date	4018

Kontrolle Gesamtzahl und Zahl aktueller Kundenversionen sind beide 10. Damit sind alle geladenen Kundenversionen aktuell. Der Kalender deckt die Jahre 2020 bis einschließlich 2030 ab.

Ergebnis Bestanden. Alle sechs Kontrollwerte entsprechen den Erwartungen.

Abbildung 3 Ergebnisnachweis aus dem SQL-Client



Schritt 4 Bestellpositionen laden

Aufgabenstellung Jede Quellposition genau einmal mit den passenden Dimensionsschlüsseln in die Faktentabelle laden.

Schritt order_item_id eindeutig absichern und die Kundenversion anhand des Bestelldatums auswählen.

Aktion und SQL Der Point-in-Time-Join verwendet inklusive Datumsgrenzen. Eine offene Version hat effective_to = NULL. revenue wird beim Einfügen durch die Datenbank generiert.

```
-- 03_load_facts.sql
BEGIN;

-- Eindeutigkeit der Quellposition sichern
CREATE UNIQUE INDEX IF NOT EXISTS uq_fact_order_items_order_item_id
ON brewflow_warehouse.fact_order_items (order_item_id);

-- Fakten mit zeitlicher Kundenzuordnung laden
INSERT INTO brewflow_warehouse.fact_order_items (
    order_item_id, order_id, customer_sk, product_sk,
    store_sk, channel_sk, date_sk, quantity, unit_price
)
SELECT
    oi.order_item_id, oi.order_id,
    dc.customer_sk, dp.product_sk, ds.store_sk,
    dch.channel_sk, dd.date_sk, oi.quantity, oi.unit_price
FROM brewflow_staging.stg_order_items oi
JOIN brewflow_staging.stg_orders o
    ON oi.order_id = o.order_id
JOIN brewflow_warehouse.dim_customer dc
    ON o.customer_id = dc.customer_id
   AND o.order_date >= dc.effective_from
   AND (dc.effective_to IS NULL
        OR o.order_date <= dc.effective_to)
JOIN brewflow_warehouse.dim_product dp
    ON oi.product_id = dp.product_id
JOIN brewflow_warehouse.dim_store ds
    ON o.store_id = ds.store_id
JOIN brewflow_warehouse.dim_channel dch
    ON o.channel = dch.channel
JOIN brewflow_warehouse.dim_date dd
    ON o.order_date = dd.calendar_date
ON CONFLICT (order_item_id) DO NOTHING;
COMMIT;
```

Kontrolle Die folgende Prüfung zählt Quellpositionen, Fakten, eindeutige Positions-IDs und fehlende Quellpositionen.

Ergebnis Das Laden ist erfolgreich. Bereits vorhandene Positions-IDs werden bei erneuter Ausführung übersprungen.

ON CONFLICT DO NOTHING aktualisiert keine bestehenden Fakten. Die bestätigte Wiederholbarkeit gilt für unveränderte Quelldaten; nachträgliche Korrekturen oder Löschungen von Quellpositionen sind damit nicht abgedeckt.

Schritt 4 Vollständigkeit der Fakten prüfen

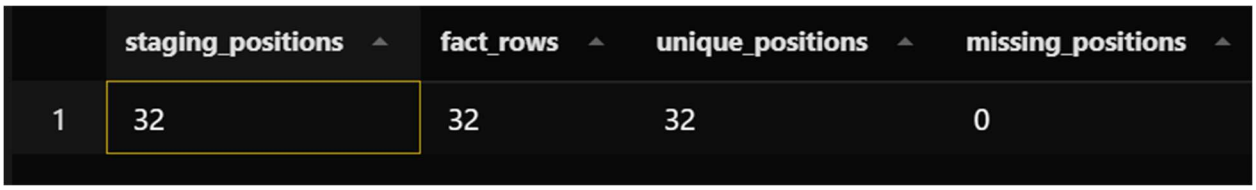
```
-- Skript 03 Kontrolle Geladene Bestellpositionen
-- Erwartet 32 / 32 / 32 / 0
SELECT
    (SELECT COUNT(*)
     FROM brewflow_staging.stg_order_items) AS staging_positions,
    COUNT(*) AS fact_rows,
    COUNT(DISTINCT f.order_item_id) AS unique_positions,
    (
        SELECT COUNT(*)
        FROM brewflow_staging.stg_order_items oi
        WHERE NOT EXISTS (
            SELECT 1
            FROM brewflow_warehouse.fact_order_items loaded
            WHERE loaded.order_item_id = oi.order_item_id
        )
    ) AS missing_positions
FROM brewflow_warehouse.fact_order_items f;
```

Kontrolle	Bestätigter Wert
staging_positions	32
fact_rows	32
unique_positions	32
missing_positions	0

Kontrolle Alle 32 Quellpositionen sind vorhanden, jede genau einmal. Keine Position wurde wegen fehlender Dimensionszuordnung beim INNER JOIN ausgelassen.

Ergebnis Bestanden. 32 Fakten, 32 eindeutige Bestellpositionen und 0 fehlende Positionen.

Abbildung 4 Ergebnisnachweis aus dem SQL-Client



	staging_positions	fact_rows	unique_positions	missing_positions
1	32	32	32	0

Schritt 5 Umsatz nach Kategorie und Kanal

Aufgabenstellung Die analytische Auswertung des Star-Schemas nach Produktkategorie und Verkaufskanal überprüfen.

Schritt Die automatisch berechneten Positionsumsätze über die zugehörigen Dimensionen aggregieren.

Aktion und SQL Zuerst nach Produktkategorie und anschließend nach Verkaufskanal gruppieren.

```
-- Ende zu Ende Prüfung Umsatz nach Produktkategorie
SELECT dp.category, SUM(f.revenue) AS total_revenue
FROM brewflow_warehouse.fact_order_items f
JOIN brewflow_warehouse.dim_product dp
    ON f.product_sk = dp.product_sk
GROUP BY dp.category
ORDER BY total_revenue DESC;

-- Ende zu Ende Prüfung Umsatz nach Verkaufskanal
SELECT dch.channel, SUM(f.revenue) AS total_revenue
FROM brewflow_warehouse.fact_order_items f
JOIN brewflow_warehouse.dim_channel dch
    ON f.channel_sk = dch.channel_sk
GROUP BY dch.channel
ORDER BY dch.channel;
```

Kategorie	**Umsatz**
Food	51,70
Coffee	51,70
Merchandise	36,00
Tea	29,60
Gesamt	169,00
Verkaufskanal	**Umsatz**
in-store	93,80
mobile-app	75,20
Gesamt	169,00

Kontrolle Die Kategorieabfrage liefert alle vier Kategorien; die Kanalabfrage beide Verkaufskanäle. Beide Gruppierungen ergeben denselben Gesamtumsatz von 169,00.

Ergebnis Bestanden. Die analytischen Abfragen liefern die erwarteten Gruppen und konsistente Umsatzsummen.

Schritt 5 Nachweise der Umsatzauswertungen

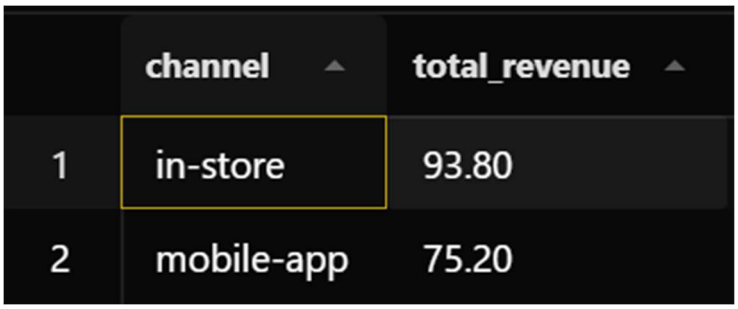
Abbildung 5 Ergebnisnachweis aus dem SQL-Client



	category	total_revenue
1	Food	51.70
2	Coffee	51.70
3	Merchandise	36.00
4	Tea	29.60

Umsatz nach Kategorie: Food und Coffee jeweils 51,70, Merchandise 36,00 und Tea 29,60.

Abbildung 6 Ergebnisnachweis aus dem SQL-Client



	channel	total_revenue
1	in-store	93.80
2	mobile-app	75.20

Umsatz nach Verkaufskanal: in-store 93,80 und mobile-app 75,20.

Schritt 5 Zweiten Durchlauf validieren

Aufgabenstellung Bestätigen, dass ein erneuter Lauf auf unveränderten Quelldaten keine zusätzlichen Datensätze erzeugt.

Schritt 01_copy_to_staging, 02_load_dimensions und 03_load_facts erneut in dieser Reihenfolge ausführen. Danach Zeilenanzahlen, eindeutige Positions-IDs und Gesamtumsatz prüfen.

Aktion und SQL Die folgende Abfrage wurde nach dem zweiten Pipeline-Durchlauf ausgeführt.

```
-- Kontrolle nach dem zweiten Pipeline Durchlauf
SELECT 'stg_customers' AS check_name, COUNT(*) AS value
FROM brewflow_staging.stg_customers
UNION ALL SELECT 'stg_stores', COUNT(*)
FROM brewflow_staging.stg_stores
UNION ALL SELECT 'stg_products', COUNT(*)
FROM brewflow_staging.stg_products
UNION ALL SELECT 'stg_orders', COUNT(*)
FROM brewflow_staging.stg_orders
UNION ALL SELECT 'stg_order_items', COUNT(*)
FROM brewflow_staging.stg_order_items
UNION ALL SELECT 'dim_customer', COUNT(*)
FROM brewflow_warehouse.dim_customer
UNION ALL SELECT 'dim_customer_current', COUNT(*)
FROM brewflow_warehouse.dim_customer WHERE is_current = TRUE
UNION ALL SELECT 'dim_product', COUNT(*)
FROM brewflow_warehouse.dim_product
UNION ALL SELECT 'dim_store', COUNT(*)
FROM brewflow_warehouse.dim_store
UNION ALL SELECT 'dim_channel', COUNT(*)
FROM brewflow_warehouse.dim_channel
UNION ALL SELECT 'dim_date', COUNT(*)
FROM brewflow_warehouse.dim_date
UNION ALL SELECT 'fact_order_items', COUNT(*)
FROM brewflow_warehouse.fact_order_items
UNION ALL SELECT 'unique_order_items', COUNT(DISTINCT order_item_id)
FROM brewflow_warehouse.fact_order_items
UNION ALL SELECT 'total_revenue', SUM(revenue)
FROM brewflow_warehouse.fact_order_items;
```

Kontrolle Staging: 10 / 5 / 8 / 22 / 32. Dimensionen einschließlich aktueller Kunden: 10 / 10 / 8 / 5 / 2 / 4018. Fakten: 32, eindeutige Positionen: 32, Gesamtumsatz: 169,00.

Ergebnis Bestanden. Die geprüften Kennzahlen bleiben nach dem zweiten Durchlauf unverändert. Der Ergebnisnachweis folgt auf der nächsten Seite.

Schritt 5 Nachweis des zweiten Durchlaufs

Abbildung 7 Ergebnisnachweis aus dem SQL-Client

	check_name	value
1	stg_customers	10
2	stg_stores	5
3	stg_products	8
4	stg_orders	22
5	stg_order_items	32
6	dim_customer	10
7	dim_customer_current	10
8	dim_product	8
9	dim_store	5
10	dim_channel	2
11	dim_date	4018
12	fact_order_items	32
13	unique_order_items	32
14	total_revenue	169.00

Abschluss und Geltungsbereich

Teil 3 ist für den unveränderten BrewFlow-Beispieldatenbestand erfolgreich abgeschlossen. Die Übertragung von der OLTP über Staging in das Warehouse wurde ausgeführt und anschließend wiederholt. Alle geforderten Zeilenanzahlen, die vier Produktkategorien, die zwei Verkaufskanäle und der Gesamtumsatz sind bestätigt.

Abschlusskriterium	**Bestätigtes Ergebnis**
Staging vollständig befüllt	10 / 5 / 8 / 22 / 32 Zeilen
Kundendimension	10 Versionen, alle is_current = TRUE
Faktentabelle	32 Zeilen und 32 eindeutige Quellpositionen
Produktkategorien	Coffee, Tea, Food und Merchandise
Wiederholter Pipeline Lauf	Unveränderte Zeilenanzahlen und Umsatz 169,00

Der SCD-Type-2-Code wurde eingerichtet; ein tatsächlicher Loyalty-Levelwechsel wurde in den vorliegenden Prüfungen nicht getestet. Die Historisierung arbeitet mit Tagesgrenzen. Mehrere unterschiedliche Levelwechsel am selben Tag benötigen eine gesonderte Behandlung.

Die Ersthistorie beruht auf der dokumentierten Annahme zum Loyalty-Level. Ein vollständiges Überschreiben im Sinne einer Bereinigung gelöschter Produkte oder Filialen wird durch die Type-1-UPDATE/INSERT-Logik nicht umgesetzt. Auch Änderungen bereits geladener Fakten und parallele Pipeline-Läufe wurden nicht geprüft.